

Constrained Control Allocation for Dynamic Ship Positioning Using an Encoder–Decoder Neural Network

Luis F. Recalde

School of Robotics Engineering
Worcester Polytechnic Institute
Worcester, Massachusetts, USA

Abstract—This work presents the implementation and evaluation of a recurrent neural network for constrained control allocation of a ship with one tunnel thruster and two azimuth thrusters. Following the encoder–decoder formulation of Skulstad et al., the encoder outputs three thrust forces and two azimuth angles. The training objective combines physical and learned force reconstruction with actuator penalties. We generate one million smooth command samples within prescribed magnitude and rate bounds. A nonlinear allocation problem implemented in CasADi and solved with IPOPT provides the optimization baseline. We evaluate force reconstruction, actuator violations, and allocation time. On a 1,024-sample evaluation segment, the median allocation times are 0.308 ms for the neural allocator and 7.955 ms for the nonlinear program. The results show that fast, accurate force reconstruction can still be accompanied by actuator rate violations.

I. INTRODUCTION

Dynamic positioning requires a vessel to generate the planar forces and yaw moment requested by a motion controller. These generalized forces must be distributed among the available thrusters while respecting their operating limits. For a vessel with azimuth thrusters, the allocation variables include both thrust magnitudes and orientations. The resulting mapping is nonlinear, and several actuator configurations can produce the same generalized force. Control allocation can therefore be formulated as an optimization problem with actuator constraints and performance requirements.

Optimization-based methods provide a direct way to include actuator saturation, rate limits, and power-related objectives. Sequential quadratic programming is one established approach to nonlinear allocation [1]. In this report, the baseline is a nonlinear program (NLP) formulated with CasADi [2] and solved using the interior-point method implemented in IPOPT [3].

An alternative is to learn the allocation map offline and evaluate a neural network during operation. We take inspiration from Skulstad et al. [4], who use an autoencoder-like recurrent network with force and actuator loss functions. The encoder outputs thruster commands, which are mapped to generalized forces through a known allocation model. This physical mapping gives the intermediate variables their meaning as actuator commands.

The objective of this report is to implement and evaluate this recurrent encoder–decoder allocator. Both allocation methods use the same signed thruster offsets, actuator ordering,

and force convention. All comparisons use prescribed force requests in open loop, without a motion controller.

II. METHODOLOGY

A. Notation and coordinate conventions

Let $\mathcal{F}_n$ denote the inertial North–East frame and $\mathcal{F}_b$ the ship-fixed frame, with x_b forward, y_b to starboard, and z_b downward. The planar pose and body velocity are

$$\boldsymbol{\eta} = [x_n \ y_n \ \psi]^\top, \quad \boldsymbol{\nu} = [u \ v \ r]^\top. \quad (1)$$

We define the generalized-force vector in sway–surge–yaw order as

$$\boldsymbol{\tau} = [\tau_{\text{sway}} \ \tau_{\text{surge}} \ \tau_{\text{yaw}}]^\top. \quad (2)$$

Forces are expressed in N, yaw moments in N m, and angles in rad unless stated otherwise.

B. Ship dynamics

We use the standard planar marine-craft model [5],

$$\dot{\boldsymbol{\eta}} = \mathbf{R}(\psi)\boldsymbol{\nu}, \quad (3)$$

$$\mathbf{M}\dot{\boldsymbol{\nu}} + \mathbf{C}(\boldsymbol{\nu})\boldsymbol{\nu} + \mathbf{D}\boldsymbol{\nu} = \mathbf{P}\boldsymbol{\tau}, \quad (4)$$

where $\mathbf{M}$, $\mathbf{C}$, and $\mathbf{D}$ are the inertia, Coriolis, and damping matrices, respectively, and

$$\mathbf{R}(\psi) = \begin{bmatrix} \cos \psi & -\sin \psi & 0 \\ \sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{P} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}. \quad (5)$$

The permutation matrix $\mathbf{P}$ converts (2) to the surge–sway–yaw order used in the dynamic equations.

The effective inertia and damping matrices have the structure

$$\mathbf{M} = \begin{bmatrix} m_{11} & 0 & 0 \\ 0 & m_{22} & m_{23} \\ 0 & m_{23} & m_{33} \end{bmatrix}, \quad \mathbf{D} = \text{diag}(d_{11}, d_{22}, d_{33}). \quad (6)$$

Table I lists the coefficients adapted from the Gunnerus model in the [Dynamic Systems Accident Simulator](#). The damping coefficients are positive under the sign convention in (4).

We define the state vector as $\mathbf{x} = [\boldsymbol{\eta}^\top, \boldsymbol{\nu}^\top]^\top$ and integrate (4) using fourth-order Runge–Kutta with a sampling time of $\Delta t = 0.05$ s.

TABLE I
EFFECTIVE SHIP-MODEL PARAMETERS, ROUNDED FOR PRESENTATION.

Parameter	Value	Unit
m_{11}	3.48061×10^5	kg
m_{22}	7.8061×10^4	kg
m_{23}	-2.30000×10^5	kg m
m_{33}	1.58304498×10^6	kg m ²
d_{11}	1.7400×10^4	N s/m
d_{22}	8.0200×10^4	N s/m
d_{33}	4.9715×10^6	N m s/rad

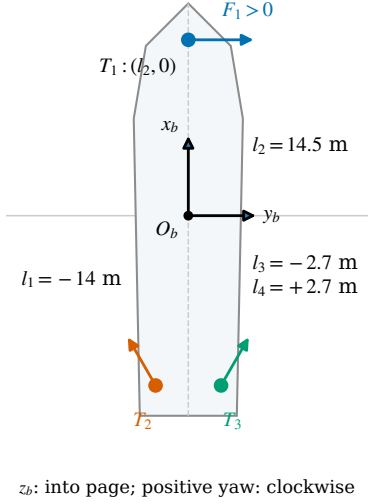


Fig. 1. Thruster locations and force directions in the ship-fixed frame.

C. Allocation model

The vessel has one tunnel thruster T_1 and two azimuth thrusters T_2, T_3 . The actuator command and thrust vectors are

$$\mathbf{u} = [F_1 \ F_2 \ \alpha_2 \ F_3 \ \alpha_3]^T, \quad \mathbf{F} = [F_1 \ F_2 \ F_3]^T. \quad (7)$$

Bold $\mathbf{u}$ denotes actuator commands, while the scalar u denotes surge velocity. The signed thruster locations relative to the body origin are

$$\mathbf{r}_1 = \begin{bmatrix} l_2 \\ 0 \end{bmatrix}, \quad \mathbf{r}_2 = \begin{bmatrix} l_1 \\ l_3 \end{bmatrix}, \quad \mathbf{r}_3 = \begin{bmatrix} l_1 \\ l_4 \end{bmatrix}, \quad (8)$$

with

$$l_1 = -14, \quad l_2 = 14.5, \quad l_3 = -2.7, \quad l_4 = 2.7 \quad [\text{m}]. \quad (9)$$

These offsets follow [4]. The thruster configuration is shown in Fig. 1.

The mapping from actuator commands to generalized forces is

$$\boldsymbol{\tau} = \mathbf{g}(\mathbf{u}) = \mathbf{B}(\boldsymbol{\alpha})\mathbf{F}, \quad \boldsymbol{\alpha} = [\alpha_2 \ \alpha_3]^T, \quad (10)$$

TABLE II
DATA-GENERATION RANGES AND PHYSICAL ACTUATOR LIMITS.

Variable	Data range	Actuator range	Rate limit
F_1 (N)	± 10000	± 30000	1000 N/s
F_2, F_3 (N)	± 5000	$[-30000, 60000]$	1000 N/s
α_2, α_3	$\pm 180^\circ$	$\pm 180^\circ$	$10^\circ/\text{s}$

where the allocation matrix is

$$\mathbf{B}(\boldsymbol{\alpha}) = \begin{bmatrix} 1 & \sin \alpha_2 & \sin \alpha_3 \\ 0 & \cos \alpha_2 & \cos \alpha_3 \\ l_2 & l_1 \sin \alpha_2 - l_3 \cos \alpha_2 & l_1 \sin \alpha_3 - l_4 \cos \alpha_3 \end{bmatrix}. \quad (11)$$

D. Actuator constraints

Table II summarizes the sampling ranges and the physical magnitude and rate limits used for allocation. Each azimuth angle also excludes the open sectors $(-100^\circ, -80^\circ)$ and $(80^\circ, 100^\circ)$; their endpoints are admissible.

E. Nonlinear allocation

The control allocation problem is formulated as an NLP. Let

$$\mathbf{e}_k = \mathbf{B}(\boldsymbol{\alpha}_k)\mathbf{F}_k - \boldsymbol{\tau}_k^d \quad (12)$$

denote the generalized-force allocation error. The cost $J_k(\mathbf{u}_k)$ penalizes this error, thrust demand, and changes in azimuth angles, subject to the actuator constraints. CasADi constructs the symbolic problem and provides its derivatives. IPOPT solves the resulting nonconvex NLP, initialized at each time step with the previous applied command.

F. Constrained random-data generation

We generate $N = 10^6$ samples of (7) within the data ranges in Table II. Each channel uses an independent random stream. Successive targets are connected by smooth quintic transitions whose durations respect the rate limits. The angles cover the full sampling range, including the forbidden sectors.

The generalized-force targets are computed using the allocation model:

$$\boldsymbol{\tau}_k^d = \mathbf{g}(\mathbf{u}_k^{\text{data}}), \quad k = 0, \dots, N-1. \quad (13)$$

We split the record chronologically into 800,000 training samples, 100,000 validation samples, and 100,000 evaluation samples. Figure 2 shows the generated commands.

III. ENCODER-DECODER DEEP NEURAL NETWORK

A. Network structure and physical intermediate variables

The network follows the recurrent encoder-decoder structure in [4]. It maps the three-dimensional desired force $\boldsymbol{\tau}_k^d$ to five physical actuator commands $\hat{\mathbf{u}}_k$, from which the decoder reconstructs a three-dimensional force $\hat{\boldsymbol{\tau}}_k$. Each half contains two 64-unit LSTM layers and a linear output layer, as shown in Fig. 3. The encoder's linear outputs are scaled by $\mathbf{S}_u = \text{diag}(10000, 5000, \pi, 5000, \pi)$ to obtain commands in N and rad. The physical force is computed separately as $\boldsymbol{\tau}_k^{\text{cmd}} = \mathbf{g}(\hat{\mathbf{u}}_k)$.

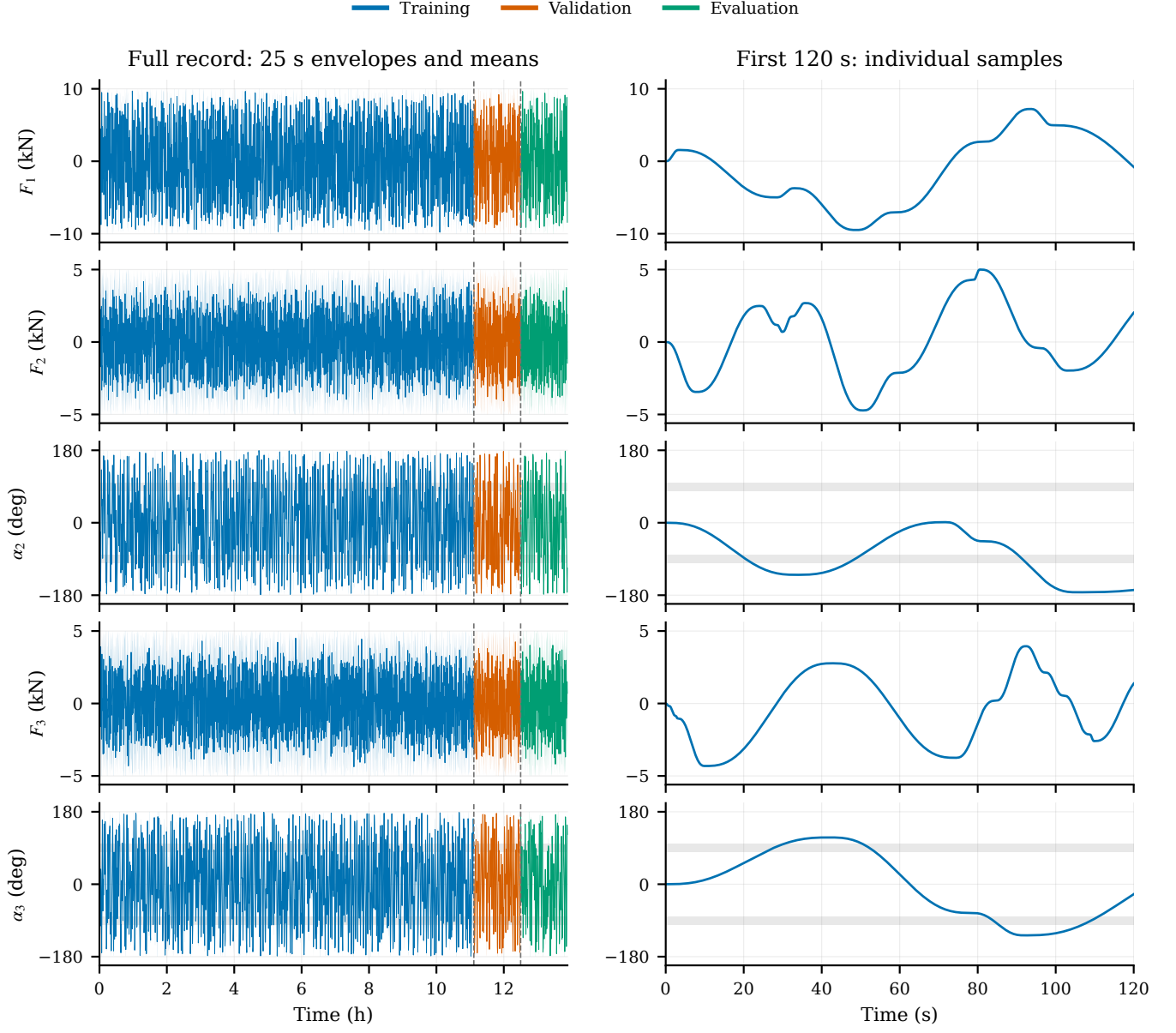


Fig. 2. One million generated command samples. Left: full-record envelopes and block means. Right: the first 120 s, showing smooth transitions. Shaded angular bands indicate forbidden sectors.

B. Cost function design

Consider a batch containing B ordered sequences of T samples, with $n = BT$. Let $\mathbf{D}_\tau = \text{diag}(\boldsymbol{\sigma}_\tau)$, where $\boldsymbol{\sigma}_\tau$ contains the force standard deviations computed from the training data.

1) *Physical force reconstruction*: The encoder must produce commands whose physical force matches the request:

$$\mathcal{L}_0 = \frac{1}{3n} \sum_{b=1}^B \sum_{k=1}^T \left\| \mathbf{D}_\tau^{-1} (\boldsymbol{\tau}_{b,k}^{\text{cmd}} - \boldsymbol{\tau}_{b,k}^d) \right\|_2^2. \quad (14)$$

This term gives the intermediate variables their physical meaning, following [4]. Accurate reconstruction by the learned

decoder alone would not ensure that the commands generate the requested force.

2) *Learned force reconstruction*: The decoder contributes

$$\mathcal{L}_1 = \frac{1}{3n} \sum_{b=1}^B \sum_{k=1}^T \left\| \mathbf{D}_\tau^{-1} (\hat{\boldsymbol{\tau}}_{b,k} - \boldsymbol{\tau}_{b,k}^d) \right\|_2^2. \quad (15)$$

Both force errors use the same training standard deviations.

3) *Command magnitudes*: Define $[x]_+ = \max(x, 0)$. The symmetric training limits are

$$\mathbf{b}_u = [30000 \quad 60000 \quad \pi \quad 60000 \quad \pi]^\top \text{ in N and rad.} \quad (16)$$

These follow the neural penalty in the reference formulation and differ from the asymmetric stern-thrust limits in Table II.

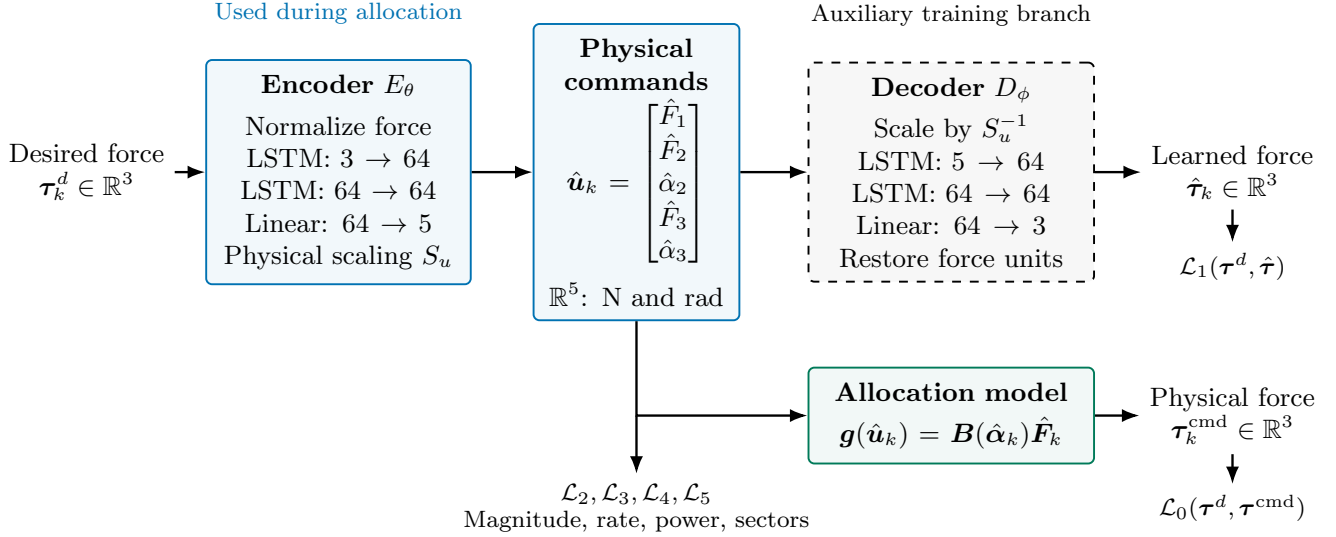


Fig. 3. Recurrent encoder-decoder with learned and physical force reconstruction branches. The encoder outputs physical thruster commands.

Using unit references $q_j = 1$ N for forces and $q_j = 1$ rad for angles,

$$\mathcal{L}_2 = \frac{1}{n} \sum_{b,k} \sum_{j=1}^5 \frac{[|\hat{u}_{b,k,j}| - b_{u,j}]_+}{q_j}. \quad (17)$$

4) *Command rate changes:* With $\Delta u_{\max,j} = \dot{u}_{\max,j} \Delta t$,

$$\mathcal{L}_3 = \frac{1}{n} \sum_{b=1}^B \sum_{k=2}^T \sum_{j=1}^5 \frac{[|\hat{u}_{b,k,j} - \hat{u}_{b,k-1,j}| - \Delta u_{\max,j}]_+}{q_j}. \quad (18)$$

Only adjacent timesteps within the same sequence are compared.

5) *Thrust demand and azimuth sectors:* The remaining terms penalize thrust demand and forbidden azimuth angles:

$$\mathcal{L}_4 = \frac{1}{n} \sum_{b,k} \sum_{i=1}^3 \left| \frac{\hat{F}_{i,b,k}}{1 \text{ N}} \right|^{3/2}, \quad (19)$$

$$\mathcal{L}_5 = \frac{1}{n} \sum_{b,k} \sum_{j \in \{2,3\}} \sum_{s=1}^2 \varphi_s(\hat{\alpha}_{j,b,k}), \quad (20)$$

where, for each forbidden interval (a_s, b_s) expressed in radians,

$$\varphi_s(\alpha) = \frac{[\min(\alpha - a_s, b_s - \alpha)]_+}{(b_s - a_s)/2}. \quad (21)$$

This piecewise linear penalty replaces the paper's indicator loss to provide training gradients.

The combined training objective is

$$\mathcal{L}(\theta, \phi) = \sum_{i=0}^5 k_i \mathcal{L}_i, \quad (22)$$

where θ and ϕ denote the encoder and decoder parameters. The scalar weights $k_i \geq 0$ assign relative importance to each term. Actuator constraints enter this objective as soft penalties.

C. Training and checkpoint selection

For the million-sample experiment, we train from a random initialization for 50 epochs using Adam [6], a learning rate of 10^{-3} , weight decay of 10^{-2} , and gradient-norm clipping at 10. The 800,000 training samples form 781 complete 1,024-sample windows and one 256-sample window. The reported losses are the six-term objective in (22). We select the checkpoint with the lowest mean validation loss, corresponding to epoch 41.

IV. EXPERIMENTAL RESULTS

A. Million-sample dataset

Figure 2 shows the generated commands, and Table IV reports their peak rates. All samples satisfy the prescribed sampling magnitude and rate limits.

B. Learning and force reconstruction

Figure 4 shows the mean total loss and three weighted components over the training epochs. Table III reports errors calculated using every sample in each split:

$$\text{RMSE}_j = \sqrt{\frac{1}{N_s} \sum_{k=1}^{N_s} (\tau_{k,j}^{\text{out}} - \tau_{k,j}^d)^2}. \quad (23)$$

Here, τ^{out} is either τ^{cmd} or $\hat{\tau}$, and N_s is the number of samples in the split. The physical reconstruction determines the force applied to the ship; the decoder provides an auxiliary learned reconstruction. Figure 5 compares both outputs with the desired forces over contiguous portions of all three splits.

Over the complete evaluation subset, the physical reconstruction RMSE is 86.09 N in sway, 64.66 N in surge, and 1452.47 N m in yaw. Relative to the training standard deviations, these correspond to 1.36%, 2.31%, and 1.61%, respectively. Training and validation errors are of comparable

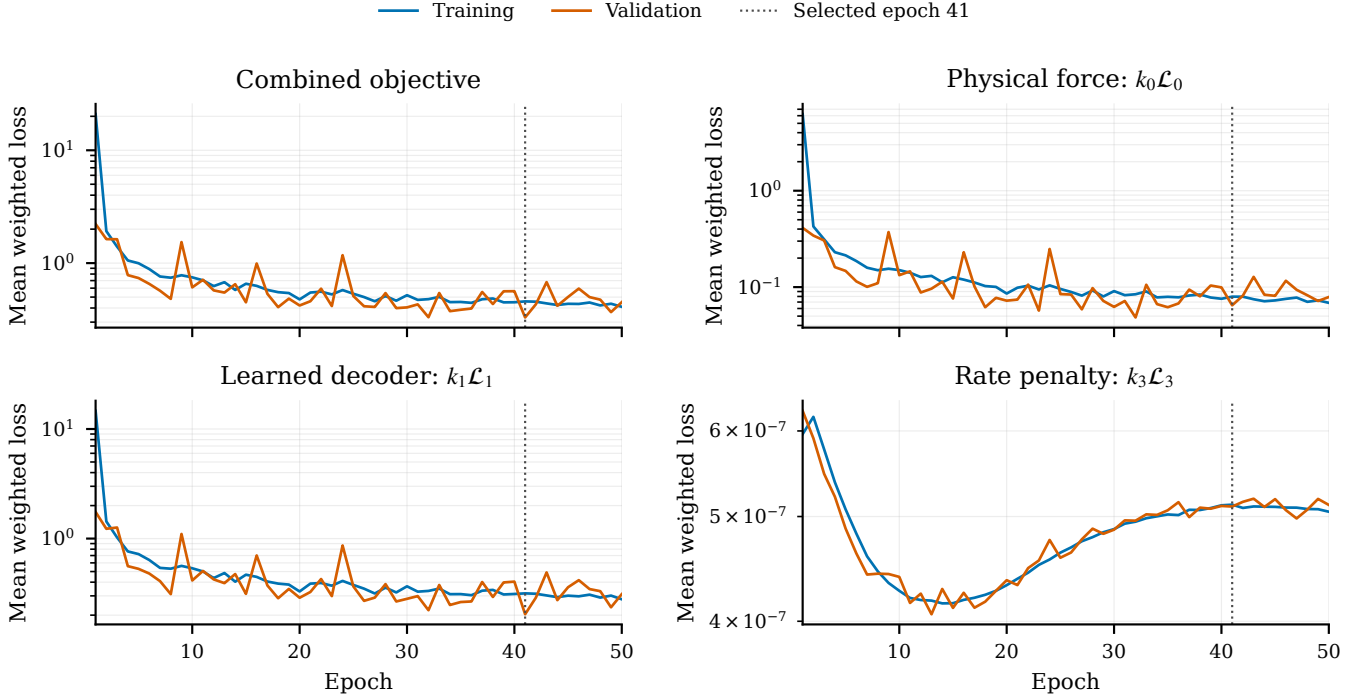


Fig. 4. Training and validation losses. The dotted line marks the selected checkpoint. The total includes all six terms; the other panels show individual weighted terms. Evaluation data are excluded from these curves.

TABLE III
FORCE RECONSTRUCTION RMSE OVER EACH COMPLETE DATA SPLIT.

Subset	Physical mapping $g(\hat{u})$			Learned decoder $\hat{\tau}$		
	Sway (N)	Surge (N)	Yaw (Nm)	Sway (N)	Surge (N)	Yaw (Nm)
Training	90.99	64.16	1482.71	160.62	73.08	2270.61
Validation	89.70	70.67	1520.25	146.43	90.58	2460.74
Evaluation	86.09	64.66	1452.47	154.57	70.56	2211.42

TABLE IV
MEASURED GENERATION RATES OVER ALL 10^6 SAMPLES.

Channel	Peak absolute rate	Allowed rate
F_1	997.386 N/s	1000 N/s
F_2	998.221 N/s	1000 N/s
α_2	9.989°/s	10°/s
F_3	995.665 N/s	1000 N/s
α_3	9.997°/s	10°/s

TABLE V
RAW ENCODER VIOLATIONS OVER EACH COMPLETE SPLIT. BOUNDS AND RATES COUNT ACTUATOR-SAMPLE VIOLATIONS; SECTORS COUNT SAMPLES WITH ANY FORBIDDEN ANGLE. NO CLIPPING IS APPLIED.

Subset	Bounds	Rates	Sectors
Training	0	40,927	0
Validation	0	5,308	0
Evaluation	0	4,502	0

magnitude. These results apply to the synthetic trajectory distribution considered here; performance under disturbances or different operating conditions remains to be evaluated.

Constraint counts for the raw commands are provided in Table V. Magnitudes are checked against the physical asymmetric bounds. Rate counts compare adjacent predictions within each split and exclude the transition from an unspecified initial actuator setting. Checks use tolerances of 10^{-3} N and 10^{-8} rad for command bounds and increments, and 10^{-8} rad at sector boundaries.

The evaluation record contains 4,502 actuator-sample rate

violations, although no magnitude or sector violations are observed. Of these rate violations, 4,150 involve the stern thrusters. The weighted rate loss remains several orders of magnitude below the force-reconstruction terms, suggesting that it has limited influence on the training objective.

C. Allocation time

On the first 1,024 evaluation requests, the median allocation times are 0.308 ms for the raw neural allocator and 7.955 ms for the NLP. Measurements use a Intel Core i9-14900HX CPU with one PyTorch thread, after ten warmup calls per method.

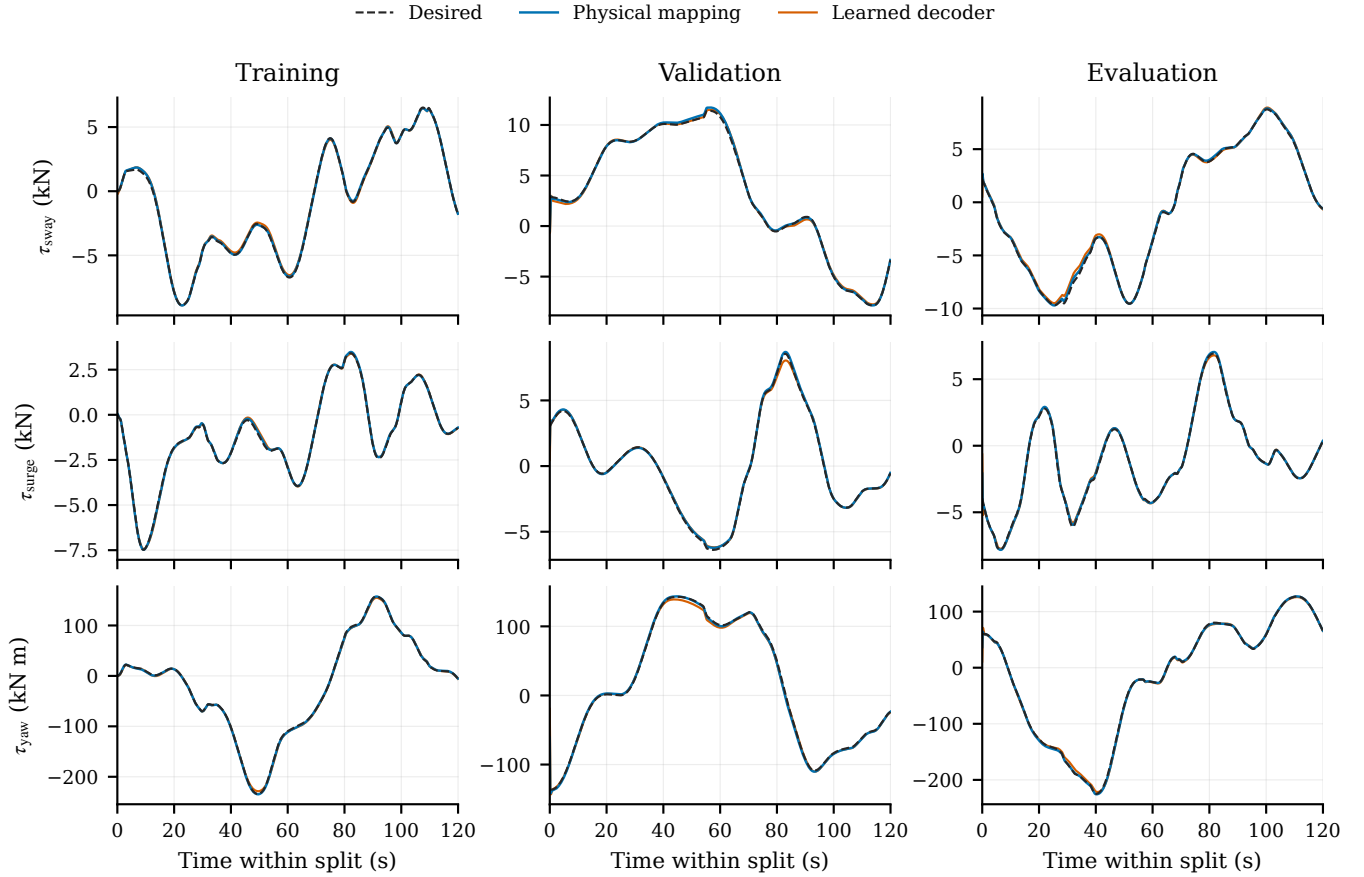


Fig. 5. Force reconstruction over the first 120 s of each split, starting from zero recurrent memory. Table III reports errors for the complete splits.

Timings cover complete allocation calls, including CasADi problem construction and solver execution for the NLP. They exclude data loading and ship integration, so the comparison concerns the current implementations.

V. CONCLUSIONS

This work implements a common ship model and thruster mapping for NLP and recurrent neural control allocation. One million smooth command samples are generated within prescribed magnitude and rate bounds and divided chronologically into training, validation, and evaluation subsets. The encoder outputs three thrust forces and two azimuth angles, while physical and learned force reconstructions contribute to the training objective.

The neural allocator achieves low force-reconstruction errors and shorter allocation times than the implemented NLP. However, rate violations remain despite the soft actuator penalties. Future work should incorporate the previous applied command and feasible command bounds into training,

then evaluate the allocator on independent trajectories and in closed-loop positioning tasks.

REFERENCES

- [1] T. A. Johansen, T. I. Fossen, and S. P. Berge, "Constrained nonlinear control allocation with singularity avoidance using sequential quadratic programming," *IEEE Transactions on Control Systems Technology*, vol. 12, no. 1, pp. 211–216, 2004.
- [2] J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl, "CasADi: A software framework for nonlinear optimization and optimal control," *Mathematical Programming Computation*, vol. 11, no. 1, pp. 1–36, 2019.
- [3] A. Wächter and L. T. Biegler, "On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming," *Mathematical Programming*, vol. 106, no. 1, pp. 25–57, 2006.
- [4] R. Skulstad, G. Li, T. I. Fossen, and H. Zhang, "Constrained control allocation for dynamic ship positioning using deep neural network," *Ocean Engineering*, vol. 279, p. 114434, 2023.
- [5] T. I. Fossen, "Fossen's marine craft model," <https://www.fossen.biz/html/marineCraftModel.html>, accessed: September 27, 2026.
- [6] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations*, 2015. [Online]. Available: <https://arxiv.org/abs/1412.6980>